



avec github comment avec une branche "version à télécharger" et une autre brache "version en développement? Comment basculer le version "en développement" vers la version "à télécharger"?

Sur GitHub, le plus courant pour gérer à la fois une version stable « à télécharger » et une version « en développement » consiste à utiliser deux branches distinctes : **main** (ou **release**) pour la version téléchargeable, et **develop** pour la version en cours de développement. [\[1\]](#) [\[2\]](#) [\[3\]](#)

Structure recommandée

- **main** : contient la version stable du projet (celle que les utilisateurs peuvent télécharger).
- **develop** : contient le code en développement, non encore prêt pour une publication.

Étapes pour mettre cela en place

1. Créer la branche de développement :

- Depuis votre terminal :

```
git checkout -b develop main  
git push -u origin develop
```

- Cela crée une branche *develop* à partir de *main* et la publie sur GitHub. [\[3\]](#) [\[4\]](#)

2. Travailler sur la branche develop :

- Vous pouvez y ajouter, modifier ou tester du code sans impacter la version stable.

3. Basculer entre les branches :

- Pour revenir à la branche stable :

```
git checkout main
```

- Pour retourner sur la branche de développement :

```
git checkout develop
```

4. Fusionner (basculer une version en développement vers la version à télécharger) :

- Lorsque la version *develop* est prête pour la publication :

```
git checkout main  
git merge develop
```

```
git push origin main
```

- Cela fusionne les changements de *develop* dans *main* pour rendre la nouvelle version disponible. ^[1] ^[3]

5. Créer une release (optionnel) :

- Une bonne pratique consiste à créer une *release* sur GitHub après la fusion, pour permettre le téléchargement d'un package ou d'un tag précis (ex. `v1.0.0`).

Résumé des commandes principales

Étape	Commande	Description
Créer la branche dev	<code>git checkout -b develop main</code>	Crée la branche de développement
Pousser la branche	<code>git push -u origin develop</code>	Publie la branche sur GitHub
Passer à main	<code>git checkout main</code>	Basculer sur la version stable
Fusionner dev → main	<code>git merge develop</code>	Bascule la version en développement vers la version téléchargeable
Publier la mise à jour	<code>git push origin main</code>	Synchronise la version stable sur GitHub

Cela permet de maintenir un flux clair entre une version de développement et une version stable, typique du workflow **GitFlow**.

✱✱

1. <https://docs.github.com/fr/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches>
2. <https://docs.github.com/fr/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/creating-and-deleting-branches-within-your-repository>
3. <https://www.varonis.com/fr/blog/git-branching>
4. <https://www.datacamp.com/fr/tutorial/github-and-git-tutorial-for-beginners>
5. <https://learn.microsoft.com/fr-fr/azure/devops/repos/git/create-branch?view=azure-devops>
6. <https://git-scm.com/book/fr/v2/Les-branches-avec-Git-Les-branches-en-bref>
7. <https://docs.github.com/fr/enterprise-server@3.14/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches>
8. <https://docs.github.com/fr/desktop/making-changes-in-a-branch/managing-branches-in-github-desktop>
9. https://github.com/betagouv/e-controle/blob/develop/docs/dev/dev_and_release_process.md
10. <https://kinsta.com/fr/blog/comment-pousser-code-github/>